# Qualität für fitnessfirst.de

Die teuren Fehler zuerst finden, und zwar früh.

# Mein Ausgangspunkt

### SO HABE ICH MICH VORBEREITET

- Die Website als Besucherin erkundet, um die Abläufe zu verstehen
- Geprüft, ob sie sich für Nutzer langsam anfühlt oder ungewöhnlich langsam ist (PageSpeed Insights, mobil)
- Geprüft, ob die Seite sich anpasst und für Handys geeignet ist
- Barrierefreiheit angeschaut (nur automatische Werte, kein vollständiger Test)
- Nichts abgeschickt, kein Login, keine Tests auf Ihrer Live-Seite

### DAS HABE ICH ANGENOMMEN

- Sie haben Testumgebungen neben der Produktion
- Zahlungsanbieter sind Dritte, darum kann man sie nicht auf der Produktion testen
- Die meisten Nutzer haben ein Handy, keinen Laptop

# So erkunde ich

Exploratives Testen: kurze Runden mit einem klaren Ziel.

**Beispiel für ein Ziel** "Den Checkout erkunden und oft die Meinung ändern, um falsche Summen zu finden."

### DAS SCHREIBE ICH JEDES MAL AUF

- Ziel und Aufbau (Gerät, Browser, Version der Seite)
- Was ich abgedeckt habe
- Fehler, mit Schritten und Screenshots
- Offene Fragen
- Was ich NICHT getestet habe

Gespeichert beim Ticket, damit das Team es sieht.

# Was ich gelernt habe: die wichtigsten Bereiche

**Risiko = wie schlimm, wenn es kaputtgeht x wie wahrscheinlich, dass es kaputtgeht**

1. Mitgliedschaft abschließen
2. Cookie-Einwilligung und Tracking
3. Pausieren, kündigen, widerrufen
4. Probetraining buchen
5. Aktionen und Angebote
6. Clubsuche
7. Mitglieder-Login
8. Kursplan
9. Magazin, Karriere

# Mein Plan für jedes Risiko

| Bereich | Hauptrisiko | Mein Plan |
|---|---|---|
| **Mitgliedschaft abschließen** | Falscher Preis, oder ein Vertrag kommt nie an | Tests für die Preisregeln, End-to-End-Tests, alle 15 Minuten eine Prüfung in der Produktion |
| **Cookie-Einwilligung und Tracking** | Marketing-Tracker laden nach "Ablehnen" | Automatische Prüfung nach "Alle ablehnen", Abstimmung mit dem Datenschutzbeauftragten |
| **Pausieren, kündigen, widerrufen** | Ein rechtlicher Ablauf funktioniert nicht mehr | End-to-End-Test bei jedem Release, Texte von Hand prüfen |
| **Probetraining buchen** | Termin gibt es nicht, Buchung erreicht den Club nie | Eine End-to-End-Buchung pro Handytyp, prüfen, dass sie ankommt |
| **Aktionen und Angebote** | Ein altes Angebot erscheint noch nach dem Ende | Tests für die Daten, anschauen, was der Cache zeigt |

**Die anderen Bereiche** leichtere Prüfungen, meistens von Hand.

# Ablauf 1: Mitgliedschaft abschließen, die Risiken

5 SCHRITTE

Club → Tarif → Extras → Ihre Daten → Zahlung

### WAS SCHIEFGEHEN KANN

- Falscher Preis: Club, Tarif, Aktion, Alter, Land
- Die Zusammenfassung passt nicht zur Auswahl
- Kunde bleibt hängen und geht
- Vertrag oder Zahlung erreicht Ihre Systeme nicht
- Rechtlicher Text fehlt oder ist falsch

**Am teuersten** ein falscher Preis oder ein Vertrag, der nie angelegt wurde.

# Ablauf 2: Probetraining buchen

3 SCHRITTE

Club → Datum und Uhrzeit → Ihre Daten

**WAS SCHIEFGEHEN KANN**

- Der gebuchte Termin existiert im Club nicht
- Die Buchung erreicht den Club nie
- Datumsfehler: heute, Wochenende, Monatswechsel, Feiertage
- Langsame Reaktion: 361 ms pro Tippen bei echten Nutzern (gut: 200 ms)

**Automatisieren** Regeln für Termine, Buchung kommt im Club an, eine End-to-End-Buchung pro Handytyp.

# Geräte, Nutzer, fremde Systeme

Geräte: zuerst der Desktop-Browser, dann die mobile Browser-Ansicht. Echte Handys: iPhone Safari, Android Chrome.

**Bildschirmbreiten** 360, 390, 768, 1280 Pixel.

**Nutzer** neue Besucher, Studenten, Menschen, die das Studio wechseln, Besucher aus Österreich, Menschen mit Tastatur oder Screenreader, langsames mobiles Internet, Menschen, die Cookies ablehnen.

### FREMDE SYSTEME, DIE ICH GESEHEN HABE

- Usercentrics (Cookie-Einwilligung)
- VWO (A/B-Tests: nicht alle sehen dieselbe Seite)
- Google Tag Manager und etwa 15 Werbedienste
- Wahrscheinlich Baqend und Cloudflare (Tempo und Cache)
- Mir unbekannt: Zahlung, Buchung, Mitgliedersystem

# Hier fange ich an: End-to-End-Tests nach Priorität

### END-TO-END-TESTS, GEBAUT IN DER REIHENFOLGE DER PRIORITÄT

1. **Priorität 1:** Mitgliedschaft abschließen, Probetraining buchen, pausieren und kündigen, Prüfung der Cookie-Einwilligung. Zuerst im Desktop-Browser gebaut.
2. **Dieselben Tests, mobile Ansicht:** die Tests der Priorität 1 noch einmal in der mobilen Browser-Ansicht, auf einem Bildschirm in Handygröße.
3. **Priorität 2:** Aktionen und Angebote, Clubsuche, Mitglieder-Login.
4. **Der Rest:** leichtere Prüfungen, meistens von Hand.

# Ablauf 1: Was ich automatisieren würde

Testpyramide: unten viele kleine, schnelle Tests, oben wenige langsame.

**Explorativ (von Hand):** neue Aktionen, ungewöhnliche Kombinationen, Österreich

**End-to-End (wenige):** einen Tarif bis zum Zahlungsschritt buchen, auf iPhone Safari und Android Chrome. Den Club mittendrin wechseln: die Zusammenfassung ändert sich.

**Optik und Barrierefreiheit:** Preiskarten und Zusammenfassung sehen richtig aus, die Tastatur funktioniert, Fehler werden vorgelesen

**API und Integration:** die richtigen Tarife und Preise pro Club. Der Vertrag erreicht das Mitgliedersystem. Zahlungsanbieter im Testmodus.

**Unit (viele):** jede Preisregel. Start- und Enddatum der Aktionen.

**Dazu in der Produktion** alle 15 Minuten eine Roboter-Prüfung der Schritte 1 bis 3.

# Was ich automatisiere und was nicht

### AUTOMATISIEREN, WENN

- der Test oft läuft, wichtig ist und das erwartete Ergebnis klar ist
- er stabil ist und wenig Pflege braucht
- Beispiel: Preisregeln (Unit-Tests)

### VON HAND LASSEN, WENN

- es neu ist oder sich noch ändert
- menschliches Urteil nötig ist: Aussehen, Wortwahl, Screenreader
- Beispiele: eine neue Aktionsseite, der Screenreader im Checkout

### WAS ICH NICHT TESTEN WÜRDE

- Fremde Dienste selbst: Zahlungsanbieter, Cookie-Werkzeug, Werbedienste. Ich teste nur unsere Seite.
- Lasttests und Sicherheitsangriffe auf die Live-Seite
- Bereiche mit niedrigem Risiko nicht in der Tiefe, zum Beispiel das Magazin: ein kurzer Blick reicht

# Der Qualitätsprozess, den ich fördern würde

Sechs Phasen, eine Frage: Was kann schiefgehen, und wie verhindern wir es?

Fehler kosten weniger, wenn wir sie früh finden.

1
**Anforderungen**
verstehen, warum

2
**Risiko**
gemeinsam entscheiden, was am wichtigsten ist

3
**Entwicklung**
früh erkunden und zu zweit arbeiten

4
**Automatisierung**
das Richtige automatisieren

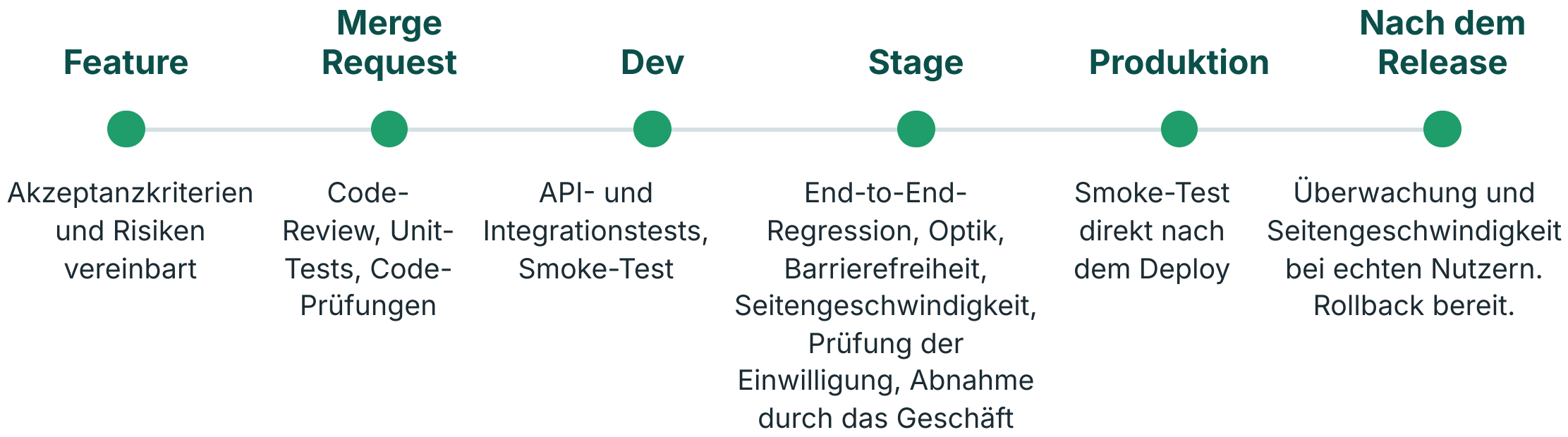
5
**Release**
"fertig" ist ein Kriterium, kein Gefühl

6
**Nach dem Release**
aus dem lernen, was durchgerutscht ist

# Qualität ist die Aufgabe des Teams

| | |
|---|---|
| **Produkt** | klare Ziele und Akzeptanzkriterien. Entscheidet gemeinsam mit dem Team über Risiken. |
| **Entwickler** | Unit- und API-Tests für ihren Code. Reparieren, was die Pipeline stoppt. |
| **Fachabteilungen** | prüfen, dass es für das Geschäft richtig ist (Abnahme). |
| **QA** | Torwächter beim Release und Hilfe für alle:<br>• hilft, Akzeptanzkriterien zu schreiben<br>• arbeitet mit Entwicklern zu zweit<br>• hält die Testfälle aktuell<br>• schult das Team im Testen |

# Von der Idee zur Live-Website




**Testdaten** keine echten Kundendaten auf Stage. Kopierte Daten werden anonymisiert.

# Wann stoppen wir ein Release?

### STOPPT DAS RELEASE

- Falscher Preis, Checkout oder Buchung kaputt
- Kündigen oder Pausieren kaputt
- "Ablehnen" bei den Cookies funktioniert nicht
- Sicherheitsproblem, oder jemand sieht die Daten einer anderen Person
- Hauptablauf mit der Tastatur nicht benutzbar

### DARF MIT BEDINGUNGEN LIVE GEHEN

- Kleiner Fehler mit einfacher Umgehung, Ticket und Termin für die Reparatur
- Problem auf einer Seite mit wenig Besuchern
- Neue Funktion mit einem Feature-Flag ausgeschaltet

**Unzuverlässige Tests** noch einmal laufen lassen. Wenn er besteht, kommt er in Quarantäne, mit Besitzer und Frist. Nie einfach ignoriert.

# Mehr als "es funktioniert"

| Bereich | Automatisch | Von Hand | Grenze (Gate) |
|---|---|---|---|
| **Barrierefreiheit** | Barrierefreiheits-Scan bei jedem Merge Request | Tastatur und Screenreader in den Hauptabläufen bei jedem Release, Experten-Prüfung einmal im Jahr | Keine kritischen Probleme im Checkout und bei der Buchung |
| **Datenschutz und Einwilligung** | Nach "Alle ablehnen" laden nur erlaubte Dienste | Texte im Banner mit dem Datenschutzbeauftragten prüfen | Jeder Marketing-Tracker nach dem Ablehnen stoppt das Release |
| **Seitengeschwindigkeit** | Lighthouse in der Pipeline, Messung der Buchungsschritte | Jede Woche die Daten echter Nutzer ansehen | Echte Nutzer: LCP 2,5 s, INP 200 ms, CLS 0,1. Pipeline: keine Seite langsamer als heute |

**Heute, echte Nutzer, mobil** alle 3 Seiten laden schnell, alle 3 reagieren zu langsam.

Mitgliedschaft INP 227 ms, Probetraining 361 ms.

# Wo KI hilft und wo nicht

### KI HILFT BEI

1. **Anforderungen:** findet Lücken und fehlende Sonderfälle in Akzeptanzkriterien
2. **Exploratives Testen:** Agenten erkunden mit einem Ziel und müssen für jedes Ergebnis einen Beweis zeigen
3. **Fehlgeschlagene Tests:** gruppiert Fehler, schlägt "echter Fehler" oder "unzuverlässig" vor

### IMMER BEI MENSCHEN

- Endgültige Priorität von Fehlern
- Freigabe oder Stopp des Releases
- Rechtliche Entscheidungen, Barrierefreiheit und Datenschutz

**Regeln** keine echten Kundendaten in KI-Werkzeugen. Jedes Ergebnis braucht einen Beweis. Die KI muss auflisten, was sie nicht getestet hat.



# Zusatzfolien

# Details zur Seitengeschwindigkeit

| Seite (mobil) | Echte Nutzer: LCP | Echte Nutzer: INP | Echte Nutzer: CLS | Laborwert |
|---|---|---|---|---|
| **Startseite** | 2 s | 208 ms | 0,03 | 43 |
| **Mitgliedschaft** | 1,1 s | 227 ms | 0,04 | 36 |
| **Probetraining** | 1,3 s | 361 ms | 0,05 | 69 |

Grenzen für "gut" (75 % der Besuche): LCP 2,5 s, INP 200 ms, CLS 0,1. Quelle: PageSpeed Insights, 27. Oktober 2026. Die Laborwerte stammen aus Einzelmessungen und schwanken.

# Testdaten und Umgebungen

- Stage: anonymisierte Kopie der Produktion, oder erzeugte Testdaten
- Feste Testkonten und Testclubs, regelmäßig zurückgesetzt
- Zahlungsanbieter im Testmodus, nie echte Zahlungen
- Fremde Systeme, die wir nicht steuern können: in API-Tests simuliert (Mock), echt nur in wenigen End-to-End-Tests auf Stage
- Test-E-Mails nur an reservierte Testdomains

# Meine KI-Testagenten (entworfen, noch nicht auf einer Live-Seite benutzt)

- 4 Agenten: Explorer, Komponente, Ablauf, API
- Regeln: nur erlaubte Testseiten, keine echten Käufe, Geheimnisse versteckt, Beweis für jedes Ergebnis
- Ergebnisse: bestanden, durchgefallen mit Priorität, oder "verdächtig" mit Begründung
- Mehrere Läufe werden zusammengeführt, weil KI-Ergebnisse von Lauf zu Lauf schwanken
- Ein P1-Fehler kann die Pipeline stoppen. Den Rest entscheiden Menschen.

# Meine ersten Wochen

| | |
|---|---|
| **Woche 1 bis 2** | das Produkt, das Team und den Release-Prozess kennenlernen. Die Fehlerhistorie lesen. |
| **Woche 3 bis 4** | Risikokarte mit dem Team, erste schnelle Erfolge (Prüfung der Einwilligung, Smoke-Tests) |
| **Monat 2 bis 3** | Regressionstests für Checkout und Buchung, Quality Gates in der Pipeline |

# Eine normale Woche

| | |
|---|---|
| **Planung** | Three Amigos bei neuen Aufgaben, Risiken vereinbart |
| **Jeden Tag** | zu zweit arbeiten, Merge Requests prüfen, kurze Fragen |
| **Während des Sprints** | explorative Runden bei neuen Funktionen |
| **Vor dem Release** | Regression auf Stage, Abnahme mit dem Geschäft |
| **Release-Tag** | Smoke-Test, Überwachung |
| **Nach dem Release** | schauen, was durchgerutscht ist, und einen Test dafür ergänzen |